

Análisis y diseño

Sistema orientado a objetos para una batalla de Digimon

Nicholas Cordón

Carné 262293

1. Alcance de la solución

El sistema lo que hace es modelar una batalla entre dos entrenadores durante cuatro rondas. Cada entrenador forma un equipo de cuatro Digimon distintos y en cada ronda, el entrenador tiene que seleccionar un Digimon que todavía no haya utilizado. El resultado se determina mediante el ataque, la defensa, la relación entre tipos y una Digievolución probabilística. La solución separa la interacción por consola, el modelo del dominio y las reglas de la batalla para aplicar encapsulamiento y relaciones entre objetos.

Resultado

Después de cuatro rondas, el programa informa cuántas rondas ganó cada entrenador y declara al vencedor de la batalla o un empate. Las rondas empatadas no suman victoria a ninguno de los participantes.

2. Requisitos funcionales

- El sistema debe solicitar y almacenar el nombre de dos entrenadores.
- El sistema debe mostrar un catálogo de Digimon con nombre, tipo, ataque, defensa y Digievolución.
- Cada entrenador debe seleccionar exactamente cuatro Digimon distintos al inicio de la partida.
- El sistema debe impedir que un entrenador agregue dos veces al mismo Digimon a su equipo.
- Cada Digimon debe tener un nombre, un tipo, un valor de ataque, un valor de defensa y una Digievolución.
- El tipo de un Digimon debe ser Fuego, Agua, Planta o Eléctrico.
- La batalla debe desarrollarse durante exactamente cuatro rondas.
- En cada ronda, cada entrenador debe seleccionar uno de los Digimon que todavía tenga disponible.
- Un Digimon utilizado debe retirarse de la lista de disponibles para impedir que se repita en rondas posteriores.
- Cada entrenador debe decidir entre atacar normalmente o intentar usar la Digievolución del Digimon seleccionado.
- Cuando se intenta una Digievolución, el sistema debe generar un número entero aleatorio entre 0 y 100.
- La Digievolución debe activarse cuando el número obtenido sea menor o igual que su probabilidad de activación.
- Una Digievolución de tipo Adult debe poder aumentar el ataque en 15 puntos.
- Una Digievolución de tipo Mega debe poder aumentar la defensa en 20 puntos.
- Una Digievolución de tipo Ultimate debe poder reducir en 10 puntos el ataque total del rival.
- El efecto de una Digievolución activada debe aplicarse en la ronda actual y en la siguiente.

- Un ataque efectivo debe recibir una bonificación de 20 puntos según las ventajas de tipo definidas.
- Un ataque débil debe recibir una penalización de 10 puntos cuando el defensor resista el tipo atacante.
- Un enfrentamiento sin ventaja o resistencia definida debe usar un modificador neutral de 0 puntos.
- El sistema debe calcular el ataque total de ambos participantes de la ronda.
- Debe ganar la ronda el entrenador cuyo Digimon obtenga el ataque total más alto.
- Si ambos ataques totales son iguales, la ronda debe terminar en empate.
- El sistema debe mantener y mostrar el número de rondas ganadas por cada entrenador.
- Al finalizar la cuarta ronda, debe ganar la batalla el entrenador con más rondas ganadas.
- Si ambos entrenadores tienen la misma cantidad de rondas ganadas, la batalla debe terminar en empate.
- El sistema debe validar que las opciones numéricas estén dentro del rango permitido y rechazar entradas no numéricas.

3. Restricciones de diseño e implementación

1. La solución debe utilizar programación orientada a objetos y relaciones entre clases.
2. Los atributos se mantienen privados para preservar el encapsulamiento; el acceso se realiza mediante métodos públicos controlados.
3. Todos los mensajes dirigidos al usuario se producen en la clase Main.
4. Las clases del dominio y de lógica no utilizan System.out.println ni System.out.print.
5. No se utiliza break dentro de ciclos while.
6. Los archivos fuente .java deben almacenarse en un repositorio público de GitHub.

4. Decisiones de implementación

4.1 Fórmula de ataque total

El enunciado indica que la batalla utiliza ataque y defensa, pero no especifica una ecuación completa. Para evitar que la defensa quede sin uso y permitir que la habilidad Mega tenga efecto, la implementación adopta la siguiente fórmula:

Fórmula

$$\text{ataqueTotal} = \text{ataque base} + \text{bonificación de ataque} + \text{modificador de tipo} - \text{defensa del rival} - \text{bonificación de defensa del rival} - \text{daño directo causado por el rival}.$$
 El resultado mínimo permitido es 0.

4.2 Uso de una Digievolución

Al escoger la acción de Digievolución, el programa conserva el ataque base del Digimon e intenta activar el efecto especial. Si la activación falla, el Digimon todavía participa con su ataque normal y los efectos que hayan quedado activos de la ronda anterior. Esta interpretación permite comparar ataques totales en todas las rondas.

4.3 Duración de efectos

El entrenador mantiene valores para la ronda actual y para la siguiente. Cuando una habilidad se activa, su cantidad se agrega a ambos grupos. Al terminar la ronda, los valores siguientes pasan a ser actuales y los acumuladores siguientes se reinician. Con ello, cada efecto dura exactamente dos rondas y los efectos simultáneos pueden acumularse de manera coherente.

5. Identificación y propósito de las clases

5.1 Tipo (enumeración)

Representa los cuatro elementos disponibles y concentra la regla de ventajas y resistencias, evitando cadenas de texto inconsistentes en el resto del sistema.

Atributos y constantes

FUEGO, AGUA, PLANTA y ELECTRICO: valores permitidos para clasificar el elemento de cada Digimon.

Métodos

calcularModificadorContra(Tipo defensor): recibe el tipo del rival y devuelve +20 para un ataque efectivo, -10 para un ataque débil o 0 para un enfrentamiento neutral.

toString(): transforma el nombre interno de la constante en un texto legible para presentarlo al usuario.

5.2 TipoEfecto (enumeración)

Define las tres categorías válidas de efecto de una Digievolución. Permite decidir qué acumulador del entrenador debe modificarse sin utilizar textos libres.

Atributos y constantes

AUMENTAR_ATAQUE: identifica una habilidad que incrementa el ataque del entrenador durante su vigencia.

AUMENTAR_DEFENSA: identifica una habilidad que incrementa la defensa aplicada contra el ataque rival.

DANIO_DIRECTO: identifica una habilidad que reduce directamente el ataque total del oponente.

5.3 Digievolucion

Modela la habilidad especial asociada a un Digimon, incluyendo su nombre, naturaleza, magnitud y posibilidad de activarse.

Atributos

nombre: nombre descriptivo de la habilidad, por ejemplo Adult, Mega o Ultimate.

tipoEfecto: indica si la habilidad aumenta ataque, aumenta defensa o provoca daño directo.

cantidad: cantidad numérica que se suma o resta cuando el efecto se activa.

probabilidadActivacion: límite entre 0 y 100 utilizado para decidir si el intento tiene éxito.

Métodos

Digievolucion(...): construye una habilidad con todos los datos necesarios para evaluar y aplicar su efecto.

lanzarActivacion(Random random): recibe el generador aleatorio de la batalla y produce un entero entre 0 y 100.

seActiva(int lanzamiento): compara el valor obtenido con la probabilidad y devuelve true cuando la habilidad se activa.

describirEfecto(): devuelve una explicación breve y legible de la consecuencia de la habilidad.

getNombre(), getTipoEfecto(), getCantidad() y getProbabilidadActivacion(): permiten consultar de forma controlada los atributos privados que necesita la lógica o la interfaz.

5.4 Digimon

Representa a un combatiente con sus estadísticas básicas, su elemento y la Digievolución que puede intentar durante una ronda.

Atributos

nombre: identifica al Digimon en el catálogo, el equipo y los resultados.

tipo: determina sus ventajas y resistencias elementales.

ataque: valor base desde el cual se calcula su ataque total.

defensa: valor base que reduce el ataque calculado por el rival.

digievolucion: objeto que contiene la habilidad especial disponible para ese Digimon.

Métodos

Digimon(...): crea un combatiente completo con nombre, tipo, estadísticas y habilidad especial.

getNombre(), getTipo(), getAtaque(), getDefensa() y getDigievolucion(): exponen únicamente la información que necesitan las demás clases sin permitir modificar directamente los atributos.

5.5 Entrenador

Representa a cada jugador, administra su equipo, controla cuáles Digimon siguen disponibles, registra las rondas ganadas y conserva los efectos temporales activos.

Atributos

nombre: identifica al jugador durante la selección, las rondas y el resultado final.

equipo: almacena los cuatro Digimon elegidos al inicio y conserva el equipo completo.

disponibles: contiene solamente los Digimon que aún pueden seleccionarse en una ronda.

rondasGanadas: acumula la cantidad de rondas obtenidas por el entrenador.

ataqueActual: bonificación de ataque vigente en la ronda que se está calculando.

ataqueSiguiente: bonificación de ataque que debe permanecer durante la ronda posterior.

defensaActual: bonificación de defensa vigente en la ronda actual.

defensaSiguiente: bonificación de defensa que debe trasladarse a la siguiente ronda.

danioActual: reducción de ataque que el efecto del entrenador aplica al rival en la ronda actual.

danioSiguiente: reducción de ataque que seguirá aplicándose al rival en la ronda posterior.

Métodos

Entrenador(String nombre): crea un jugador sin equipo, sin victorias y sin efectos activos.

agregarDigimon(Digimon digimon): verifica que el nombre no esté repetido y que el equipo tenga menos de cuatro integrantes; devuelve true cuando logra agregarlo.

usarDigimon(int indice): retira y devuelve el Digimon seleccionado de la lista de disponibles para evitar que vuelva a usarse.

aplicarDigievolucion(Digievolucion digievolucion): consulta el tipo y la cantidad del efecto y los agrega a los acumuladores de la ronda actual y siguiente.

avanzarRonda(): traslada los efectos siguientes a los acumuladores actuales y reinicia los siguientes para controlar la duración.

sumarRondaGanada(): incrementa en una unidad el marcador del entrenador.

getNombre(): devuelve el nombre del entrenador.

getEquipo(): devuelve una vista no modificable del equipo completo para proteger la colección interna.

getDigimonesDisponibles(): devuelve una vista no modificable de los combatientes que aún pueden elegirse.

getCantidadEquipo(): informa cuántos Digimon han sido agregados durante la selección inicial.

getRondasGanadas(): devuelve el marcador actual.

getAtaqueActual(), getDefensaActual() y getDanioActual(): proporcionan a Batalla los modificadores temporales necesarios para calcular una ronda.

5.6 ResultadoRonda

Es un objeto de transferencia que transporta desde Batalla hacia Main todos los datos que deben mostrarse, sin obligar a la clase de lógica a imprimir mensajes.

Atributos

ataqueTotal1 y ataqueTotal2: guardan los resultados numéricos de ambos participantes.

modificadorTipo1 y modificadorTipo2: registran la bonificación o penalización elemental aplicada a cada Digimon.

lanzamiento1 y lanzamiento2: guardan el número aleatorio obtenido; usan -1 cuando no se intentó una habilidad.

habilidadActivada1 y habilidadActivada2: indican si el intento de cada entrenador tuvo éxito.

ganador: codifica 0 para empate, 1 para el primer entrenador y 2 para el segundo.

Métodos

ResultadoRonda(...): recibe y conserva todos los datos producidos al resolver la ronda.

Métodos get e is: permiten que Main consulte ataques, modificadores, lanzamientos, activaciones y ganador para construir los mensajes de salida.

5.7 Batalla

Concentra las reglas del combate. Evalúa habilidades, calcula modificadores, obtiene ataques totales, determina el ganador y actualiza el estado de los entrenadores sin realizar interacción por consola.

Atributos

random: generador compartido por todas las rondas para obtener los valores de activación de las habilidades.

Métodos

Batalla(): construye el controlador de combate e inicializa el generador aleatorio.

jugarRonda(...): recibe entrenadores, Digimon y acciones; intenta las habilidades, aplica efectos, calcula ambos ataques, asigna la victoria, avanza los efectos y devuelve un ResultadoRonda.

calcularAtaqueTotal(...): método privado que combina ataque, modificadores, defensa y daño directo, y limita el resultado mínimo a cero.

5.8 Main

Es la clase principal y la única responsable de comunicarse con el usuario. Crea los objetos iniciales, coordina la selección y las cuatro rondas, valida entradas y presenta resultados.

Atributos

SCANNER: lector estático compartido por los métodos de Main para recibir entradas desde la consola.

Métodos

main(String[] args): punto de entrada; solicita nombres, crea entrenadores, forma equipos, ejecuta cuatro rondas y presenta el resultado final.

crearCatalogo(): construye y devuelve los Digimon disponibles con sus estadísticas y Digievoluciones.

seleccionarEquipo(...): repite la selección hasta que el entrenador tenga cuatro Digimon distintos.

mostrarCatalogo(...): presenta todos los datos relevantes de los combatientes disponibles.

seleccionarParaRonda(...): muestra los Digimon no utilizados, valida la opción y obtiene el combatiente elegido.

seleccionarAccion(...): muestra las acciones disponibles y devuelve true cuando el usuario escoge usar la habilidad.

mostrarResultado(...): presenta activaciones, ataques totales, ganador de la ronda y marcador acumulado.

mostrarIntentoHabilidad(...): informa el lanzamiento aleatorio y si la Digievolución se activó.

mostrarGanadorFinal(...): compara las rondas ganadas y presenta al vencedor o el empate final.

formatoConSigno(int valor): convierte los modificadores positivos en textos con signo + para facilitar su lectura.

leerTexto(String mensaje): solicita un texto y no termina hasta recibir un valor que no esté vacío.

leerEnteroEnRango(...): solicita, convierte y valida un entero dentro del intervalo permitido, controlando entradas inválidas.

6. Relaciones entre clases

Las relaciones permiten distribuir responsabilidades y evitan concentrar toda la solución en la clase principal.

Origen	Relación	Destino	Justificación
Entrenador	Agregación 1 a 4	Digimon	Administra un equipo de cuatro combatientes, que pueden existir como objetos independientes del entrenador.
Digimon	Composición 1 a 1	Digievolucion	Cada combatiente se construye con una habilidad especial que forma parte de su definición.
Digimon	Asociación	Tipo	Consulta su tipo para calcular ventajas y resistencias.
Digievolucion	Asociación	TipoEfecto	Usa la categoría para decidir qué estadística o ataque rival debe modificar.
Batalla	Dependencia	Entrenador y Digimon	Recibe estos objetos para resolver una ronda y actualizar el marcador y los efectos.
Batalla	Creación	ResultadoRonda	Construye un resultado con los datos que Main necesita presentar.
Main	Dependencia	Clases del sistema	Crea, coordina y consulta objetos, pero delega las reglas de negocio.

7. Encapsulamiento y distribución de responsabilidades

Los atributos del modelo se declaran privados y, cuando su valor no debe cambiar después de construir el objeto, se marcan final. Los métodos get permiten lectura controlada. Entrenador devuelve vistas no modificables de sus colecciones para impedir cambios externos que evadan las reglas de selección. Batalla contiene las reglas de cálculo; las clases de datos no imprimen mensajes; y Main concentra exclusivamente la entrada y salida. Esta separación mantiene una responsabilidad principal por clase y facilita modificar reglas sin reescribir la interfaz.

8. Correspondencia entre requisitos y componentes

Requisitos	Componente principal	Cobertura
RF-01 a RF-04	Main y Entrenador	Registro de participantes y creación validada de equipos.
RF-05 y RF-06	Digimon, Tipo y Digievolucion	Representación completa de combatientes y elementos permitidos.
RF-07 a RF-10	Main y Entrenador	Control de cuatro rondas, disponibilidad y selección de acciones.
RF-11 a RF-16	Digievolucion, TipoEfecto y Entrenador	Activación probabilística, aplicación y duración de efectos.
RF-17 a RF-20	Tipo y Batalla	Modificadores elementales y cálculo de ataque total.
RF-21 a RF-25	Batalla, ResultadoRonda y Entrenador	Ganadores, empates y marcador final.
RF-26	Main	Validación de números, rangos y textos no vacíos.

9. Diseño UML

Los diagramas siguientes emplean la notación de visibilidad UML: “+” para miembros públicos y “-” para miembros privados. La composición se muestra con rombo lleno, la agregación con rombo vacío y las dependencias con línea discontinua.

Diagrama UML 1. Modelo del dominio y relaciones estructurales

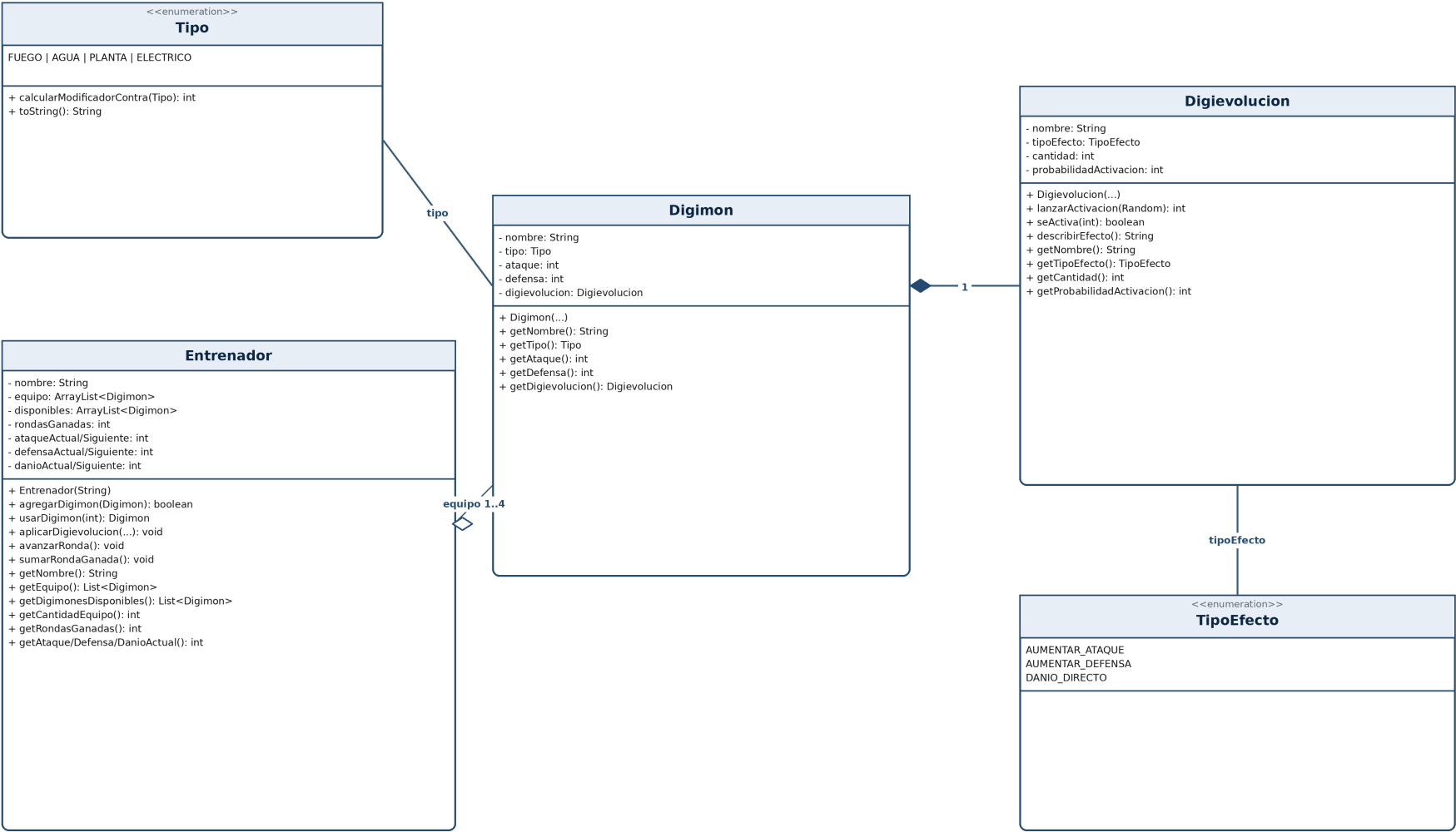


Diagrama UML 2. Coordinación, cálculo y transferencia del resultado

